

INTERNATIONAL CONFERENCE ON RECENT TRENDS IN ADVANCED COMPUTING
2019, ICRTAC 2019

Multi Criteria based Resource Score Heuristic for Cloud Workflow Scheduling

Chitra S

Maharani Lakshmi Ammanni College for Women Autonomous
Bangalore-560012, India

Abstract

Scheduling scientific workflows modelled by Directed Acyclic Graphs (DAG) is an NP complete problem. Cloud computing provides reliable Quality of Service defined in terms of Service Level Agreements (SLA). To schedule scientific workflows in cloud environment, where resources are shared, it is important to manage the cloud resources efficiently by maximizing utilization. The dynamic nature of cloud resources, due to sharing, heterogeneity, virtualization and workload variations offer a host of challenges in terms of resource availability and performance. This may have a significant impact on task execution times and data transfer times, thus introducing delay in overall execution time called makespan. In this paper, Multi Criteria based Resource Score Heuristic for Cloud Workflow Scheduling is proposed, with an objective of minimizing makespan considering the probability of temporal availability of resources in a cloud computing environment. To choose the best virtual machine with optimal value of maximum resource availability and minimum task execution time, a method of compensatory aggregation of conflicting criteria, is used for scoring each resource. The simulation results demonstrate that the proposed heuristic, can generate schedules with better makespan minimization and is found to be more reliable since resource availability factor is considered while mapping tasks to virtual machines.

© 2019 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the INTERNATIONAL CONFERENCE ON RECENT TRENDS IN ADVANCED COMPUTING 2019.

Keywords: Directed Acyclic Graphs, Makespan, Reliability, Resource Availability, Workflow Scheduling

*Corresponding author. Tel.: +91 9243001122

E-mail address: chitraravi.cr@gmail.com

1. Introduction

Cloud computing uses Internet technologies to provide a host of services by maintaining massive infrastructure and acquiring resources dynamically to support variable workloads [1]. Applications in the field of science such as computational biology, chemistry, genetics, seismology, astronomy, are highly data and compute intensive. Scientific workflows are divided into several smaller and simpler tasks that could be distributed to several compute resources for execution in the cloud. They require large amount of bandwidth and powerful computational resources for execution. Active and competent scheduling algorithm is very vital to use the cloud resource model for executing such data and compute intensive applications. IaaS clouds offer an easily accessible, flexible, and scalable compute, storage and networking infrastructure for the deployment of large-scale scientific workflows [2]. They enable workflow management systems to access a shared compute infrastructure on-demand and on a pay-per-use basis. With the flexibility and abundance of resources, there is a need for resource provisioning and resource allocation strategy with scheduling aiming at obtaining a suitable mapping between tasks and resources. Several virtual infrastructure managers have a dynamic resource allocation feature to continuously monitor utilisation across resource pools and then reallocate the available resources among virtual machines as per the application needs [3]. Resource Provisioning manages resources and improves QoS parameters for ensuring guaranteed performance for applications. In Software-Defined Networking (SDN)-enabled clouds, resource provisioning is possible by reconfiguring network flows dynamically adapting to network traffics [14].

The dynamic nature of the resources in a cloud environment, is due to both the network connectivity and computational nodes. As the resources are not dedicated, and can be used by other users simultaneously, there are load variations on resources. The local tasks have higher priority than global tasks. The fluctuations in the resource availability (actual computing speed of the host and links bandwidth) due to local load on the resource and competition from other users can cause the original schedule to become sub optimal. This may have a significant impact when scheduling workflows on clouds, introducing delays. If, for instance, load of the processor increases, the execution time of the tasks assigned to it will increase. Further, with sudden increase in link load, there is an increase in the data transfer times between hosts where dependent tasks are to be executed. Also, dynamic workflow traffic arrivals cause CPU load variations and network bandwidth variations that can be monitored using Resource monitoring tools, affect resource availability factor. This has a large impact on scheduling performance. Dynamic failure of resources with a given mean time between failures (MTBF) /mean time to repair (MTTR), can be considered while determining the earliest finish time of task on virtual machines, by adding MTBF/MTTR to the Earliest Start Time (EST) and Task Execution Time (TET) while making scheduling decisions. Many scheduling policies depend on the estimation of task execution times on different VMs to decide on task to VM mapping. This estimate is done based on the VMs computing capacity and if this capacity is always assumed to be optimal during the planning phase, the actual task execution will most probably take longer, and the task will be delayed. An online incremental learning approach can be used to predict scientific workflow task execution times on cloud in the form of time series data [13]. The delay of tasks is also propagated to the child processes thus contributing to the workflow execution finish time. There are certain existing static task scheduling algorithms in literature such as HEFT [3], ECTS [4] that schedule tasks based on earliest finish time. They do not consider the temporal availability of resources in making scheduling decisions.

The proposed Multi Criteria based Resource Score Heuristic for Cloud Workflow Scheduling (MCRS) is based on multi criteria decision making method TOPSIS [7] Score for Resource selection. The heuristic is developed with the objective of minimizing makespan considering the probability of temporal availability of physical resources to the Virtual Machines in a cloud computing environment. After initial schedule is made based on priority listing of tasks, runtime refinement of task to VM mapping to adapt to the dynamic changes in cloud resource availability is done. The multiple conflicting criteria depicting various resource and application parameters can be monitored and used resulting in quality schedules. To choose the best computing resource, a method of compensatory aggregation of criteria is used for scoring each resource. The simulation results demonstrate that the proposed MCRS heuristic that generates high-quality schedules by achieving better makespan as compared to existing algorithms that assumes full resource availability while mapping tasks to virtual machines.

2.Related Work

In Heterogeneous Earliest Finish Time (HEFT) algorithm [3] there are two phases, Task prioritization and processor selection. Priority list is formed by arranging tasks in descending order of upward rank. The selected task is assigned

to processor on which earliest finish time is minimum, using insertion-based approach. The variation in resource availability factor due to dynamic nature of cloud is not considered. Multi-Criteria Decision Making (MCDM) is used to find the best option among all feasible alternatives consisting of multiple incompatible criteria on QoS parameters [5]. Godse and Mulik proposed a method to rank various business functionalities of SaaS services using Analytic Hierarchy Process, considering functionality, architecture, usability, vendor reputation, and cost [6]. Although this is an efficient approach, it does not consider the uncertainty of decision in determining pairwise comparison. The Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS) is an MCDM method used to determine the best alternative [7]. TOPSIS method is simple and best option than other MCDM methods, when conflicting criteria and very large and very small value parameters are provided. There is no limitation in the number of alternatives and criteria to be added.

3. MCRS Algorithm

In this section, the Multi Criteria based Resource Score Heuristic for Cloud Workflow Scheduling (MCRS) is discussed. Scientific application workflows represent complex computational problems to be executed in distributed computing environments. They are modeled as Directed Acyclic Graphs (DAGs). Node depicts task and directed edge between two nodes represent dependencies [10]. Data in workflows are processed using files. Computation costs are calculated based on Task length and CPU speed in MIPS. The mode of sharing these files affects performance as it influences cost and makespan [10]. When multiple VMs concurrently run on the same physical host, they share the available physical resources, including CPUs, disks, memory, network adapter. The performance variation in the underlying hardware, fluctuations in provisioning and deprovisioning times of Virtual Machines (VM), network bandwidth variations cause network delay and hence affect communication cost due to data transfer time delays. This adds to the resource availability variations while tasks are being executed on VMs. This can be accounted for by a probability of resource availability factor that is included as a constraint in the selection and allocation of a suitable VM for execution of tasks.

The proposed heuristic for scheduling scientific workflow applications in a cloud environment, with the goal of makespan minimization, considers the dynamic provisioning and heterogeneity of computing resources as well as VM performance variation. After initial static schedule, runtime refinement of task to VM mapping to adapt to the dynamic changes in cloud scenario is adopted, where the multiple conflicting criteria are handled by allocating resources to the tasks based on Score using Technique for Order of Prioritisation by Similarity to Ideal Solution (TOPSIS). The desired parameters of higher resource availability and lower Earliest Finish Time are the criteria considered in selecting resource for allocation. The favourable values are minimum earliest finish time and maximum resource availability. The adverse condition is that of having values of minimum resource availability and maximum earliest finish time. The resource score can be calculated for each resource, balancing these conditions.

There are two phases: Task prioritizing phase and Processor Selection phase. In the first phase, a priority list is formed by ordering tasks in the decreasing order of upward rank or level. Tie-break is random. In the second phase, Resource Allocation is based on score calculated using TOPSIS method that considers resource availability and Earliest Finish Time as the two decision making criteria to generate task schedule output. The positive ideal solution is having highest possible resource availability factor and least possible Earliest finish time as these are the desired values for earliest task completion with reliability [7]. The negative ideal solution is the lowest possible resource availability factor and highest possible Earliest finish time. A set of alternatives are compared by identifying weights for each criterion. The scores are normalized for each criterion. The distance of each alternative with positive ideal alternative is calculated and a set is formed. The distance of each alternative from negative ideal solution is calculated and another set is formed. The ideal solution is the collection of ideal scores in all attributes considered. The similarity to the worst condition is determined. The alternatives are ranked according to similarity to worst condition. So, the VM with the maximum score is chosen.

The steps involved in the MCRS heuristic are:

1. Submit the list of workflow tasks. i.e. $T=T_1, T_2, T_3, \dots, T_n$.
2. Get the accessible virtual resources from data centre. i.e. $MM=VM_1, VM_2, VM_3, \dots, VM_n$.
3. An evaluation matrix consisting of 'm' Virtual machines $v_1, v_2, \dots, v_m$ and 'n' criteria x_1 and x_2 (resource availability factor and earliest finish time so $n=2$) for scoring is created with elements denoted as x_{ij} , where i is each task and j is each virtual machine.

4. Weights of the criteria are given so that $\sum_{j=1}^n w_j = 1$. Since there are two criteria considered for scoring, each weight is 0.5.
5. Obtain the final scores of the VMs for Task Allocation i.e. Resource Selection
6. Since the numbers x_{ij} present values of different criteria with different measuring units, we consider the given weights w_j of the criteria x_j
 - i. First, the measuring numbers x_{ij} of the criteria x_j are replaced with the normalized or relative numbers $r_{ij} = \frac{x_{ij}}{\sum_{i=1}^m x_{ij}^2}$
 - ii. According to the share $w_j x_j$ of the criteria x_j , the normalized numbers r_{ij} are replaced with weighted normalized numbers $a_{ij} = w_j r_{ij}$
 - iii. For our objective of minimizing makespan considering resource availability factor, the positive ideal solution consists of the set of maximum resource availability factor and minimum earliest finish time for each virtual machine. Let positive ideal solution set be $A^* = (t_1^*, t_2^*)$
 - iv. Likewise, the negative ideal solution consists of the set of minimum resource availability factor and maximum earliest finish time for each virtual machine. Let the negative ideal solution set be $A^0 = (t_1^0, t_2^0)$
 - v. To calculate the geometric distance between the target alternative (virtual machine) and the worst condition, we use the formulae $d_{iw} = \sqrt{\sum_{j=1}^n (t_{ij} - t_{wj})^2}$ for each vm $i = 1, \dots, m$
 - vi. To calculate the geometric distance between the target alternative (virtual machine) and the best condition, we use the formulae $d_{ib} = \sqrt{\sum_{j=1}^n (t_{ij} - t_{bj})^2}$ for each vm $i = 1, \dots, m$
 - vii. The similarity to the worst condition is calculated as:
 - a. $s_{iw} = \frac{d_{iw}}{d_{iw} + d_{ib}}$ for each vm $i = 1, \dots, m$; $0 \leq s_{iw} \leq 1$
 - b. $s_{iw} = 1$ if the alternative gives the best solution
 - c. $s_{iw} = 0$ if the alternative gives the worst solution
7. The Virtual machine with maximum score s_{iw} is chosen to be allocated to the task
8. While task_list has tasks to schedule, repeat steps 6, 7 and 8 otherwise return the mapping of resources to tasks.
9. Pick the maximum score VM from the VM list.
10. Find the earliest finish time of the task on that specific VM.
11. Select the next task from the list.
12. If all tasks are scheduled, then their mapping to VMs is returned for the simulation.

ALGORITHM MCERS

BEGIN MCERS

```

//T is set of tasks in the workflow and M is set of VMs available from data center
//Ravail - probability of VM availability matrix
//n - number of criteria for scoring resource
//x1 and x2 are the criteria and wj is weightage for each criterion such that  $\sum_{j=1}^n w_j = 1$ 
For each task
  Calculate ranku //Task Prioritization
End for
For each task
  For each VM
    Generate randomly  $Ravail(t_i, vm_j)$ 
  Endfor
Endfor
RdyTskLst ← startnode
While RdyTskLst is NOT NULL do
  ti ← Node in the RdyTskLst with maximum ranku
  For each vm
    Calculate Averagebandwidth =  $\sum vm_j / M$ 
    Calculate Computation cost( $t_i, vm_j$ ) = task size/vmmips =  $t_i / vm_j$ 
  Calculate communication cost as data transfer cost from parent to child task node

```

```

End for
For each VM
    Calculate VM score using TOPSIS method
Endfor
Choose the VM with maximum score
Allocate chosen VM to current task  $t_i$ 
Update  $RdyTskLst$ 
Endwhile
END MCRS

```

MCRS generates high quality schedule using optimization technique by adopting heuristic approach using the knowledge about the characteristics of the cloud (number of VMs, resource availability factor, start time, end time) as well as the workflow application (task priority and task length) to find a schedule that meets the user's QoS requirements.

4. Simulation Study

The Workflowsim toolkit was used for simulating scientific workflows in distributed environments [12]. This is developed above the Cloudsim simulator[8][9]. Scientific application workflows representing complex computational problems are modelled as Directed Acyclic Graphs (DAGs). A number of scientific workflows from different domains, that are compute intensive, or data intensive or I/O intensive were considered for studying the performance of the proposed heuristic and comparing with existing HEFT algorithm that assumes 100% resource availability during scheduling decisions not considering resource availability factor. The scientific workflows are shown in Figure 1. The Epigenomics workflow [11] of Bioinformatics field shown in Figure 1(a) is compute intensive and automates execution of various genome sequencing operations. Montage [11] workflow shown in Figure 1(b) is an I/O intensive astronomy application that creates custom sky mosaics. Cybershake [11] is an example for data and memory intensive scientific workflow that characterizes earthquake hazards and is shown in Figure 1(c). LIGO Inspiral [11] workflow is used in the field of Astrophysics and is shown in Figure 1(d). Each type of workflow differs in topological structures [11].

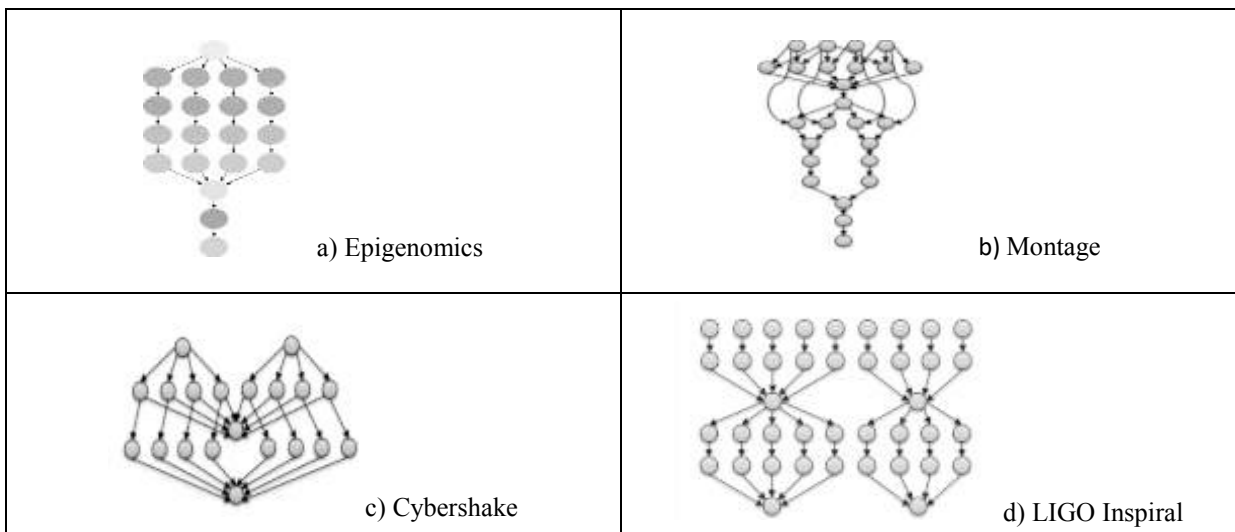


Figure 1: Scientific Workflows

The MCRS, which is the static scheduling heuristic adapting to the dynamic environment by considering resource availability factor with a probabilistic approach, is submitted to the workflow engine which in turn submits to the underlying cloud simulator where the job is executed.

5. Results and Discussions

The performance of the proposed algorithm is studied by testing it on several application workflows

modelled by DAGs and compared with the HEFT algorithm for the same resource availability scenario. The two algorithms were executed for the same workflow and the schedules were recorded. The workflows used were Epigenomics (compute intensive), Montage, Cybershake (data and memory intensive), and LIGO Inspiral. The makespan for varied resource availability factors is observed and averaged. Figure 2 shows graphs comparing makespan of MCRS and HEFT algorithms for various scientific workflows. In static task scheduling algorithms such as HEFT, since temporal factor of resource availability is not considered, there is a delay which increases makespan. The performance of our algorithm is much better in terms of reduced makespan compared to HEFT algorithm.

Test Case 1: The performance of MCRS is tested by comparison of makespan of MCRS and HEFT for Epigenomics Workflow graphs of 24,46,100 and 997 nodes. For all cases, MCRS algorithm gives lesser makespan meeting the objective of minimizing makespan as shown in Figure 2. For Epigenomics workflow of 24 nodes, the percentage reduction in makespan is 23.6%. For Epigenomics workflow of 46 nodes, the percentage reduction in makespan is found to be 18.7%, for 100 nodes Epigenomics workflow, it is 19.6% reduction in makespan and for 997 nodes Epigenomics workflow it is 22.13%.

Test Case 2: For Montage graph, the performance of MCRS is studied by comparing makespan of MCRS and HEFT for graphs of 30,60,100 and 1000 task nodes. For all cases, MCRS performs better as shown in Figure 2. For Montage graph of 30 nodes, the percentage reduction in makespan of MCRS compared to HEFT is 8.72%. For Montage graph of 60 nodes, the percentage reduction in makespan is 23.4%. For Montage graph of size 100 nodes, the percentage reduction is 27.66% and for 1000 node graph it is 41.86%.

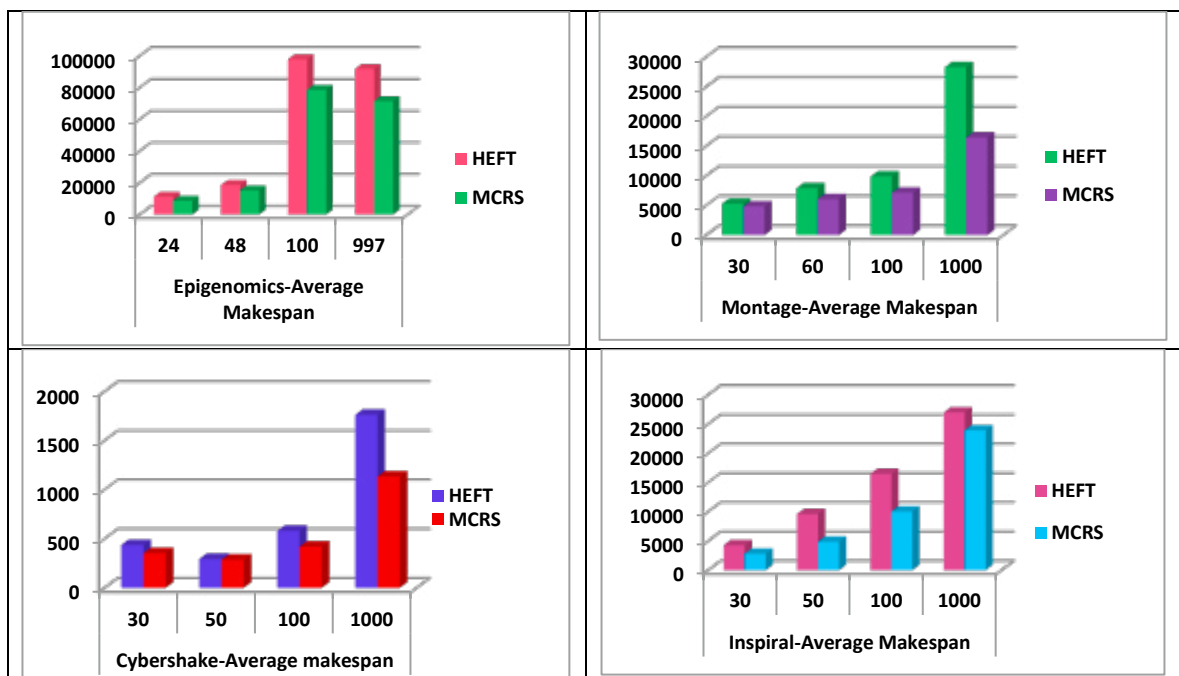


Figure 2: Comparison of Makespan of MCRS and HEFT for various scientific workflows

Test Case 3: MCRS heuristic performance is tested using Cybershake Workflow in comparison with HEFT using graphs of 30,60,100 and 1000 task nodes. It is found that for all sizes of workflows, MCRS algorithm gives lesser makespan meeting the objective of minimizing makespan as shown in Figure 2. For Cybershake graph of size 30 nodes, percentage improvement is 18.94% while for 50 node graphs, it is 23.12%. for 100 node graphs, the percentage reduction in makespan is 27.78% and for 1000 nodes Cybershake graph, it is 35.75%.

Test Case 4: The performance of MCRS in comparison with HEFT for Inspiral workflow graphs of 30,50,100 and 1000 task nodes is found to be better resulting in lesser makespan meeting the objective of makespan minimization, as shown in Figure 2. For Inspiral graph of size 30 nodes, the percentage reduction in makespan is 34.15%, for 50

node graphs, it is 49.36% for 100 node graph it is 39.17% and for 1000 node graph, it is 11.34%.

6. Conclusion

In cloud environment, there are fluctuations in resource availability due to shared resources, heterogeneity and vastly varying workloads. There are uncertainties in performance of Virtual machines due to network connectivity as well as computational nodes. The dynamic nature of cloud resources offers a host of challenges in terms of resource availability and performance. Effective Workflow scheduling with reliability is an important factor to be considered. In this paper, a novel heuristic called Multi Criteria based Resource Score Heuristic for Cloud Workflow Scheduling based on TOPSIS Score for Resource selection is proposed and tested for performance. The heuristic is developed with the objective of minimizing makespan considering the probability of temporal availability of physical resources to the Virtual Machines in a cloud computing environment, thus providing better reliability. In Cloud scenario, when requests are dynamic, after initial static schedule, runtime refinement of task to VM mapping is adopted. MCRS was tested for performance in terms of makespan minimization, by comparing with existing HEFT algorithm, using the scientific workflows Epigenomics, Montage, CyberShake and LIGO Inspiral of various task node sizes. For Epigenomics workflow of 24,46,100,997 nodes, the percentage reduction in makespan ranges from 18.7% to 23.6%. For Montage workflow of 30,60,100,1000 nodes, the percentage reduction in makespan of MCRS compared to HEFT ranges between 8.72% and 41.86%. For Cybershake graph of size 30,50,100,1000 nodes, percentage improvement in makespan is 18.94% to 35.75%. For LIGO Inspiral graph of size 30,50,100,1000 nodes, the percentage reduction in makespan is ranging between 11.34% to 49.36%. In all test cases, it is found that the proposed MCRS algorithm performs better than existing algorithm in minimizing makespan. Since this algorithm considers the resource availability factor along with Earliest Finish Time for scoring the resource for selection and allocation, this algorithm is more reliable in terms of completion of workflow execution.

References

- [1] Dan C. Marinescu, Morgan Kaufmann, (2013), "Cloud Computing Theory and Practice," Elsevier
- [2] Rajkumar Buyya, James Broberg, Andrzej Goscinski, (2010), "Cloud Computing: Principles and Paradigms", Wiley Publications
- [3] Haluk Topcuoglu, Salim Hariri and Min-You Wu (2002), "Performance Effective and Low Complexity Task Scheduling for Heterogeneous Computing", IEEE Transactions on Parallel and Distributed Systems, Vol.13, No.3: 260-274
- [4] R. Eswari and S. Nickolas, (2011), "A Level-wise Priority Based Task Scheduling for Heterogeneous Systems", International Journal of Information and Education Technology, Vol. 1, No. 5, p.p. 371-376 ISSN: 2010-3689, DOI: 10.7763/IJiet. 2011.V1.60 (ECTS)
- [5] Chandrashekar Jatoth, G. R. Gangadharan, Ugo Fiore, RajkumarBuyya, (2018), "SELCLLOUD: a hybrid multi-criteria decision-making model for selection of cloud services", Soft Computing (2019) 23:4701–4715, Published online: 9 March 2018 © Springer-Verlag GmbH Germany, part of Springer Nature
- [6] Godse M, Mulik S, (2009), "An approach for selecting software-as-a-service (SaaS) product. In: Proceedings of the IEEE international conference on cloud computing, pp 155–158
- [7] Majid Behzadian, S. Khan Mohammadi Otaghsara, Morteza Yazdani, Joshua Ignatius, (2012), "A state-of-the-art survey of TOPSIS applications", Expert Systems with Applications, 13051–13069, Elsevier
- [8] Rajkumar Buyya¹, Rajiv Ranjan² and Rodrigo N. Calheiros¹, (2009), "Modelling and Simulation of Scalable Cloud Computing Environments and the CloudSim Toolkit: Challenges and Opportunities"
- [9] Rodrigo N. Calheiros, Rajiv Ranjan, Anton Beloglazov, Cesar A. F. De Rose and Rajkumar Buyya¹, (2011), "CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," SOFTWARE – PRACTICE AND EXPERIENCE 41:23–50 Published online 24 August 2010 in Wiley Online Library (wileyonlinelibrary.com). DOI: 10.1002/spe.995
- [10] Maria A. Rodriguez, Rajkumar Buyya, (2017), "A Taxonomy and Survey on Scheduling Algorithms for Scientific Workflows in IaaS Cloud Computing Environments", CONCURRENCY AND COMPUTATION: PRACTICE AND EXPERIENCE
- [11] Maria A. Rodriguez and Rajkumar Buyya (2014), "Deadline Based Resource Provisioning and Scheduling Algorithm for Scientific Workflows on Clouds" IEEE Transactions on Cloud Computing, Volume 2, Issue 2, Pages: 222-235
- [12] Weiwei Chen, Ewa Deelman, (2012), "WorkflowSim : A Toolkit for Simulating Scientific Workflows in Distributed Environments" The 8th IEEE International Conference on eScience 2012 (eScience 2012), Chicago, Oct 8-12
- [13] Muhammad Hafizhuddin Hilman, Maria A. Rodriguez and Rajkumar Buyya, (2018), "Task Runtime Prediction in Scientific Workflows Using an Online Incremental Learning Approach", IEEE/ACM 11th International Conference on Utility and Cloud Computing (UCC)
- [14] Jungmin Son, Rajkumar Buyya, (2019), "Priority-Aware VM Allocation and Network Bandwidth Provisioning in Software-Defined Networking (SDN)-Enabled Clouds", IEEE Transactions on Sustainable Computing, Vol. 4, No. 1